CSIS 3380 · MODULE 6 · LAB

useState, Step by Step

A hands-on lab on React's first hook

A React component is a function, and React calls that function again every time it needs to redraw the component. So how does a component *remember* anything, such as how many times a button was clicked? A normal variable is created fresh on every call. The answer is **state**, and in a function component you get state from the `useState` hook.

In this lab you work through the files in the `usestate` folder. You start with a one-button counter, break it on purpose to see why state is needed, grow it into a counter with a step size, and finish with a three-level component tree where the state lives at the top and the buttons sit at the bottom.

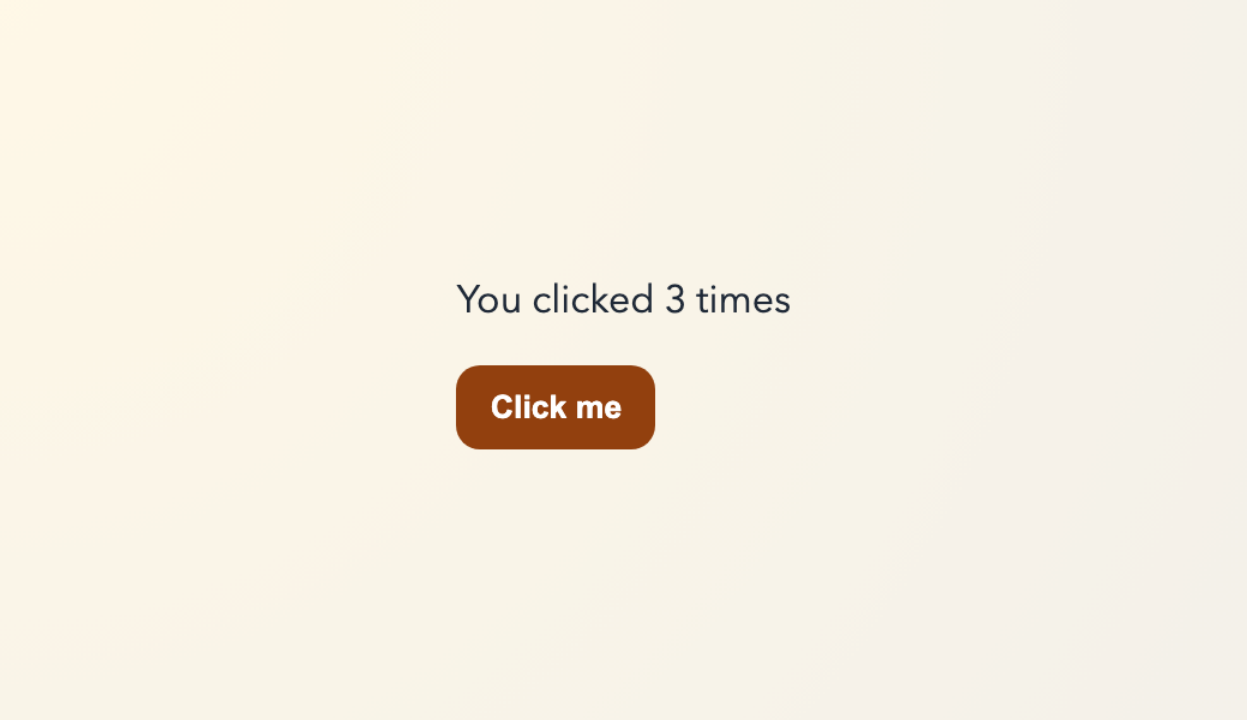


Figure 1 — the first counter from app-cdn.js, after three clicks

> HOW TO USE THIS HANDOUT
> Type the code yourself; reading it is not the same as writing it. Every step that changes code ends with a **Checkpoint** folder in `code/checkpoints/`. If you fall behind, copy that folder's files over yours and carry on.

CONTENTS

Step	Topic	File	Minutes
00	Set up, and how the page loads React	`index.html`	8
01	Your first state variable	`app-cdn.js`	9
02	Why a plain variable is not enough	`app-cdn.js`	8
03	Three buttons and the updater function	`app.js`	8
04	Pass a value, or pass an updater?	`app.js`	10
05	A second piece of state: the step size	`app.js`	10
06	Lifting state up	`lifting-state.js`	9
07	Send a new callback down the tree	`lifting-state.js`	8
08	Wrap-up, challenge and self-check		5
	Total		**75**

Step 00 — Set up, and how the page loads React

Goal: the starter project running in Chrome, and an idea of what each `<script>` tag does.

1. Unzip `usestate.zip` (or copy `code/starter/`) and open the `usestate` folder in VS Code. You should see seven files:

File	What it is
`index.html`	the page for the counter; loads `app.js`
`app.js`	a counter with Increment, Decrement and Reset
`app-cdn.js`	a smaller counter with one button
`styles.css`	styles for both counters
`lifting-state.html`	a second page; loads `lifting-state.js`
`lifting-state.js`	the same counter split over three components

File	What it is
`lifting-state.css`	extra styles for the second page

2. Open `index.html`. Near the bottom are four script tags:

```
<script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
<script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
<script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
<script type="text/babel" src="./app.js"></script>
```

The first two load **React** and **ReactDOM** from a CDN, as global variables named `React` and `ReactDOM`. The third loads **Babel**, which translates JSX into plain JavaScript inside the browser. The fourth is your code. Its `type="text/babel"` tells the browser to leave it alone, so that Babel can translate it first.

3. Right-click `index.html` and choose **Open with Live Server**. The folder's `.vscode/settings.json` sets the port to **5501**. Open Chrome DevTools (**F12**) on the **Console** tab and keep it open for the whole lab.

> **WHY LIVE SERVER, NOT DOUBLE-CLICK?**
> Babel fetches `app.js` with a network request. A page opened straight from disk (`file://…`) is not allowed to do that, so the page stays blank. Always open these pages through Live Server.

4. Open `app.js` and look at the first line:

```
const { useState } = React;
```

This takes the `useState` function out of the global `React` object. In a Vite project you would write `import { useState } from "react";` instead. Everything else in this lab is the same in both.

> **CHECKPOINT 00**
> `code/starter/`

Step 01 — Your first state variable

Goal: the one-button counter in `app-cdn.js` running, and every part of the `useState` line named.

1. In `index.html`, change the last script tag so that it loads the smaller counter:

```
<script type="text/babel" src="./app-cdn.js"></script>
```

2. Save. Live Server reloads the page. Click **Click me** a few times and watch the sentence change (Figure 1).
3. Open `app-cdn.js` and find the important line:

```
const [count, setCount] = useState(0);
```

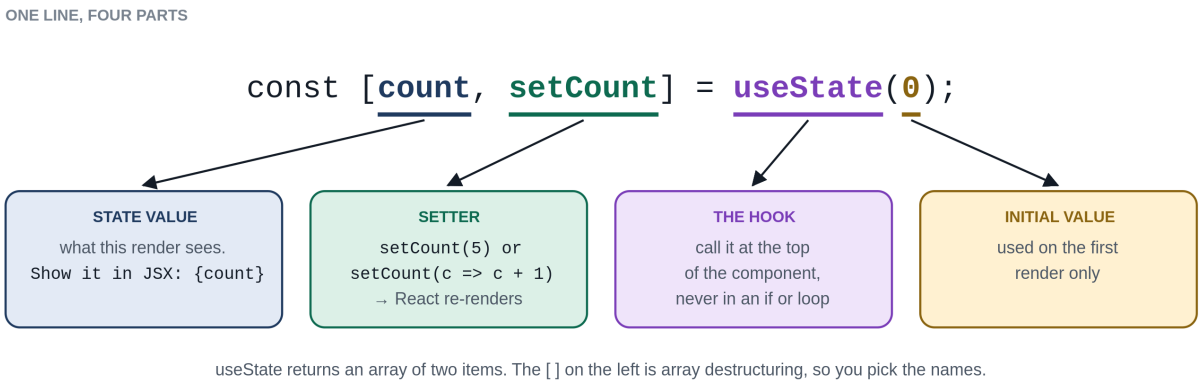


Figure 2 — the four parts of a useState line

`useState(0)` asks React for one piece of state that starts at `0`. React gives back an array with two items: the **current value** and a **setter function** that changes it. The square brackets on the left are array destructuring, so the names are up to you. The convention is `thing` and `setThing`.

4. Now read the button:

```
<button onClick={() => setCount(count + 1)}>
    Click me
</button>
```

Clicking does not change `count` directly. It calls `setCount` with the new value. React stores the new value, then calls `Counter()` again. This time `useState(0)` ignores its `0` and returns the stored value, so the new render shows the new number.

> **THE RENDER CYCLE IN ONE SENTENCE**
> You call the setter → React stores the new value → React calls your component again → the JSX it returns uses the new value → React updates only the parts of the page that changed.

> **CHECKPOINT 01**
> `code/checkpoints/step-01-first-counter/`

Step 02 — Why a plain variable is not enough

Goal: see with your own eyes that a normal variable cannot do the job, then watch React re-render.

1. **Experiment.** Replace the body of `Counter` in `app-cdn.js` with this version, which uses a plain variable and no hook:

```
function Counter() {
    console.log("Counter rendered");
    let count = 0;
    return (
        <div>
            <p>You clicked {count} times</p>
            <button onClick={() => { count = count + 1; console.log("count is now",
count); }}>
                Click me
            </button>
        </div>
    );
}
```

2. Save and click **Click me** three times. Look at the Console (top half of Figure 3). The variable *does* change: 1, 2, 3. The page still says **You clicked 0 times**, and **Counter rendered** appeared only once.

Two things went wrong. Changing a normal variable does not tell React anything, so React never calls `Counter()` again. And even if it did, `let count = 0` would run again and start from zero.

3. Undo the experiment: put back the original `useState` version of `Counter`. Then add one line right after the `useState` line:

```
const [count, setCount] = useState(0);
console.log("Counter rendered, count =", count);
```

4. Save, clear the Console (the 🚫 icon) and click three times. Now there is one **Counter rendered** line per click, each with the new value (bottom half of Figure 3).

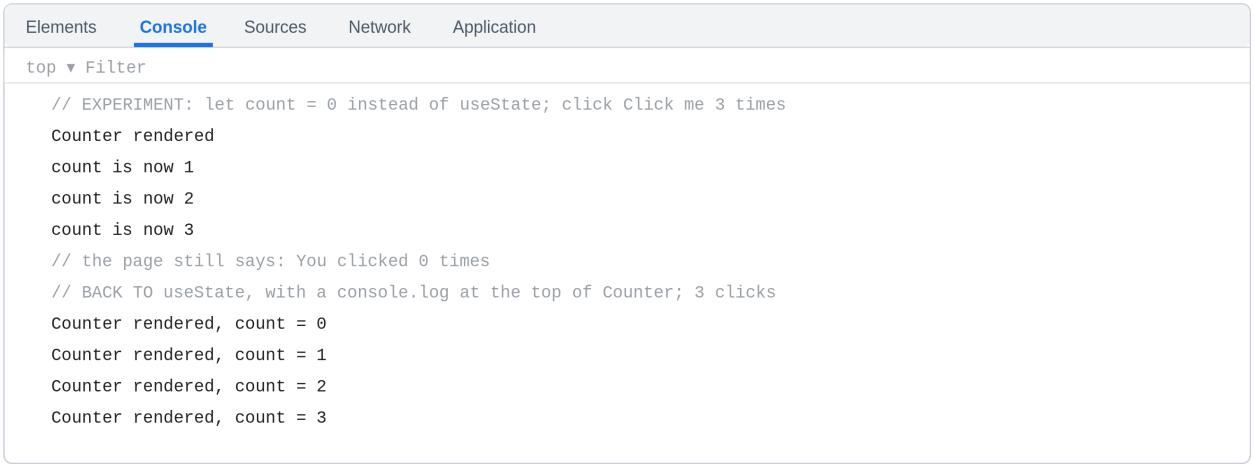


Figure 3 — a plain variable versus useState, as seen in the Console

> **STATE IS WHAT REACT REMEMBERS FOR YOU**
> React keeps the value outside your function, between calls. The setter is the only way to change it, because calling the setter is what tells React to render again.

> **CHECKPOINT 02**
> `code/checkpoints/step-02-render-log/`

Step 03 — Three buttons and the updater function

Goal: the full counter in `app.js` running, and the two ways of calling a setter.

1. In `index.html`, switch the script tag back to `app.js`:

```
<script type="text/babel" src="./app.js"></script>
```

2. In `app.js`, add the same kind of log line after `useState`:

```
const [count, setCount] = useState(0);
console.log("App rendered, count =", count);
```

3. Save and try all three buttons (Figure 4).

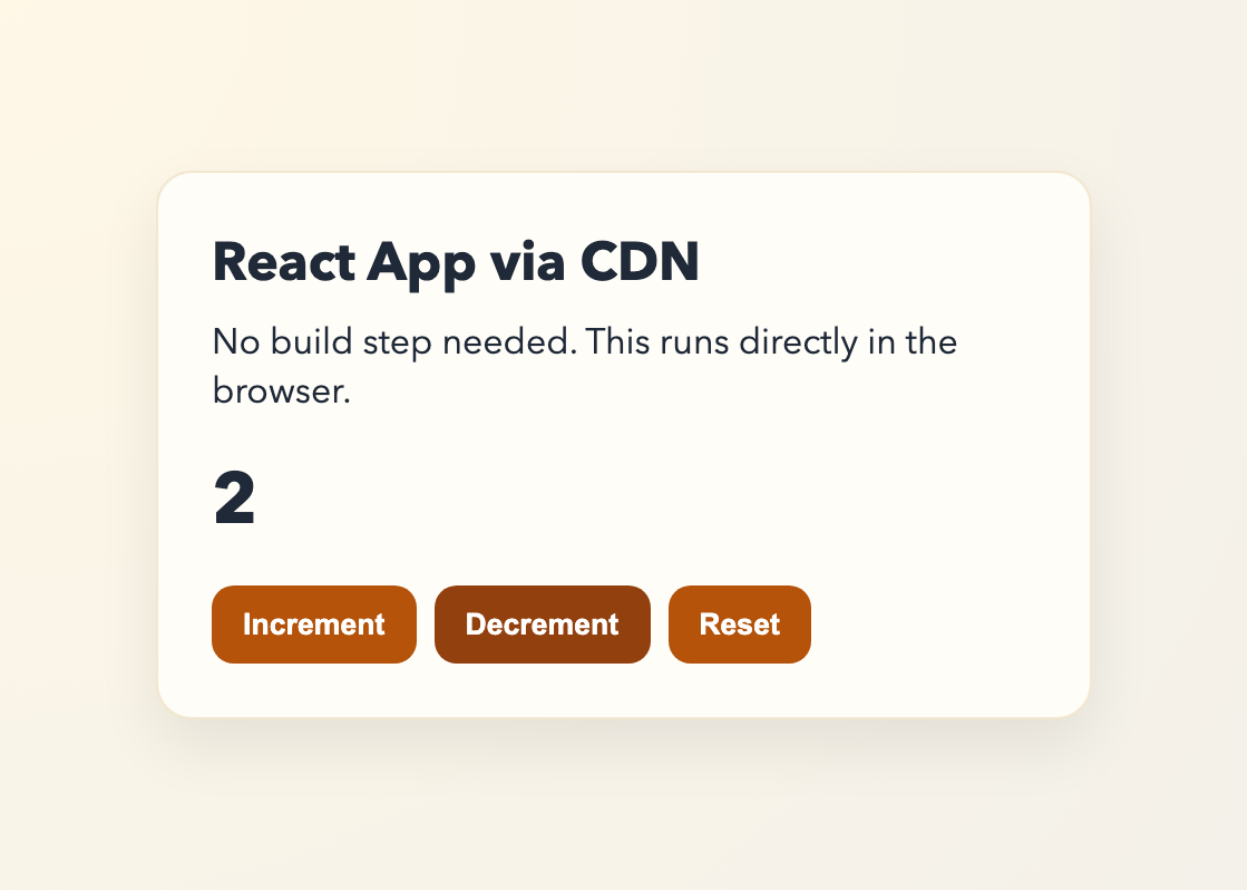


Figure 4 — app.js after three Increments and one Decrement

4. Read the three handlers:

```
<button onClick={() => setCount((c) => c + 1)}>Increment</button>
<button onClick={() => setCount((c) => c - 1)}>Decrement</button>
<button onClick={() => setCount(0)}>Reset</button>
```

Reset passes a **value**: “make it 0”. The new value does not depend on the old one.

Increment and **Decrement** pass a **function**, called an *updater*: “take whatever the count is and add 1”. React calls the updater with the latest value, `c`, and stores what it returns. The name `c` is your choice; `prev` is also common.

5. Notice that `onClick={() => setCount(0)}` wraps the call in an arrow function. If you wrote `onClick={setCount(0)}`, the setter would run *while rendering*, which triggers another render, which runs it again. React stops with a **Too many re-renders** error in the Console and the page goes blank. Try it if you like, then put the arrow back.

> **CHECKPOINT 03**
> `code/checkpoints/step-03-three-buttons/`

Step 04 — Pass a value, or pass an updater?

Goal: a **+3** button, and the reason the updater form exists.

1. **Experiment.** Add a fourth button after Reset that calls the setter three times, passing a value each time:

```
<button
  onClick={() => {
    setCount(count + 1);
    setCount(count + 1);
    setCount(count + 1);
  }}
>
  +3
</button>
```

2. Save, clear the Console and click **+3**. The count goes up by **1**, not 3.

Inside one render, `count` is a fixed number, a snapshot. If it was 0 when you clicked, all three lines say `setCount(0 + 1)`. You asked three times for “make it 1” (Figure 5, left).

3. Change the three lines to the updater form:

```
setCount((c) => c + 1);
setCount((c) => c + 1);
setCount((c) => c + 1);
```

4. Save and click **+3** again. It now adds 3. React lines the updaters up and hands each one the result of the one before (Figure 5, right).

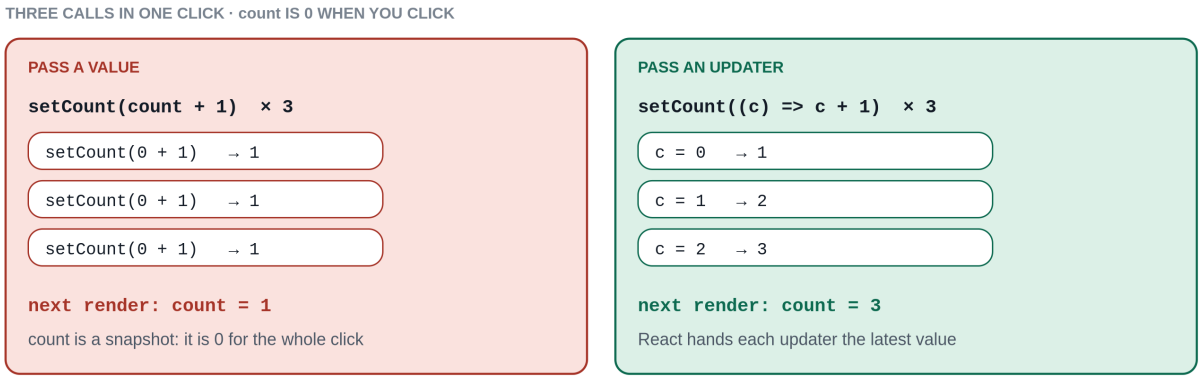


Figure 5 — three value calls give 1; three updater calls give 3

5. Look at the Console after one **+3** click. There is only **one** new **App rendered** line, not three. React *batches* every state update made during one event and renders once at the end.

> **RULE OF THUMB**
> If the new value is calculated from the old one, pass an updater: `setCount((c) => c + 1)`. If it is a fixed value, pass the value: `setCount(0)`.

> **CHECKPOINT 04**
> `code/checkpoints/step-04-updater-function/`

Step 05 — A second piece of state: the step size

Goal: a **Step size** box that controls how much Increment and Decrement change the count.

1. A component can call `useState` as many times as it needs. Each call is a separate, independent piece of state. Add a second one under the first:

```
const [count, setCount] = useState(0);
const [step, setStep] = useState(1);
console.log("App rendered, count =", count, "step =", step);
```

2. Add a labelled number box between the count and the buttons:

```
<div className="count">{count}</div>
<label className="step">
  Step size
  <input
    type="number"
    min="1"
    value={step}
    onChange={(e) => setStep(Number(e.target.value))}
  />
</label>
```

This is a **controlled input**. Its `value` always comes from state, and every keystroke calls `onChange`, which updates the state, which re-renders the box with the new value. An input always hands you a string, even a number box, so `Number(…)` turns `"5"` into `5`. Without it, `0 + "5"` would give the string `"05"`.

3. Use `step` in the two handlers, and delete the **+3** button; the step size replaces it:

```
<button onClick={() => setCount((c) => c + step)}>Increment</button>
<button onClick={() => setCount((c) => c - step)}>Decrement</button>
<button onClick={() => setCount(0)}>Reset</button>
```

4. Add these rules to the end of `styles.css`:

```
.step {
  display: flex;
  align-items: center;
  gap: 8px;
  margin-bottom: 16px;
}

.step input {
  width: 64px;
  padding: 6px 8px;
  font: inherit;
}
```

5. Save. Type **5** in the box and click Increment three times (Figure 6). In the Console, typing in the box logs a render with a new `step` and the same `count`. Setting one piece of state leaves the others alone.

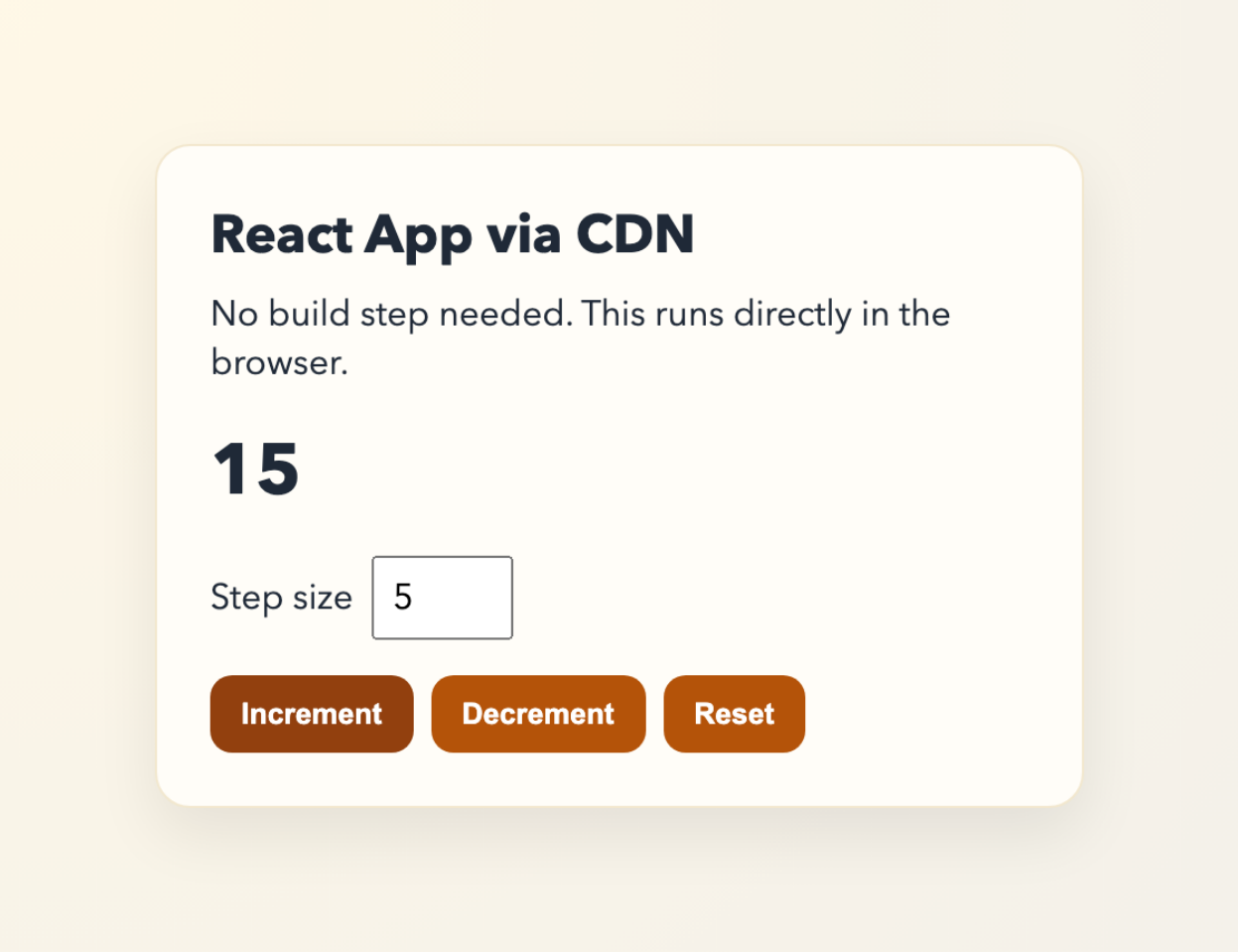


Figure 6 — step size 5, after three Increments

> **TRY IT (1 MIN)**
> Click Reset. The count goes to 0 but the step stays at 5. Should Reset put the step back to 1 too? If you think so, add `setStep(1);` to the Reset handler. Both updates are batched into one render.

> **CHECKPOINT 05**
> `code/checkpoints/step-05-step-size/`

Step 06 — Lifting state up

Goal: understand where state should live when several components need it.

1. Open `lifting-state.html` with Live Server. It is the same counter, now built from three components nested inside each other (Figure 7). Each coloured box is one component.

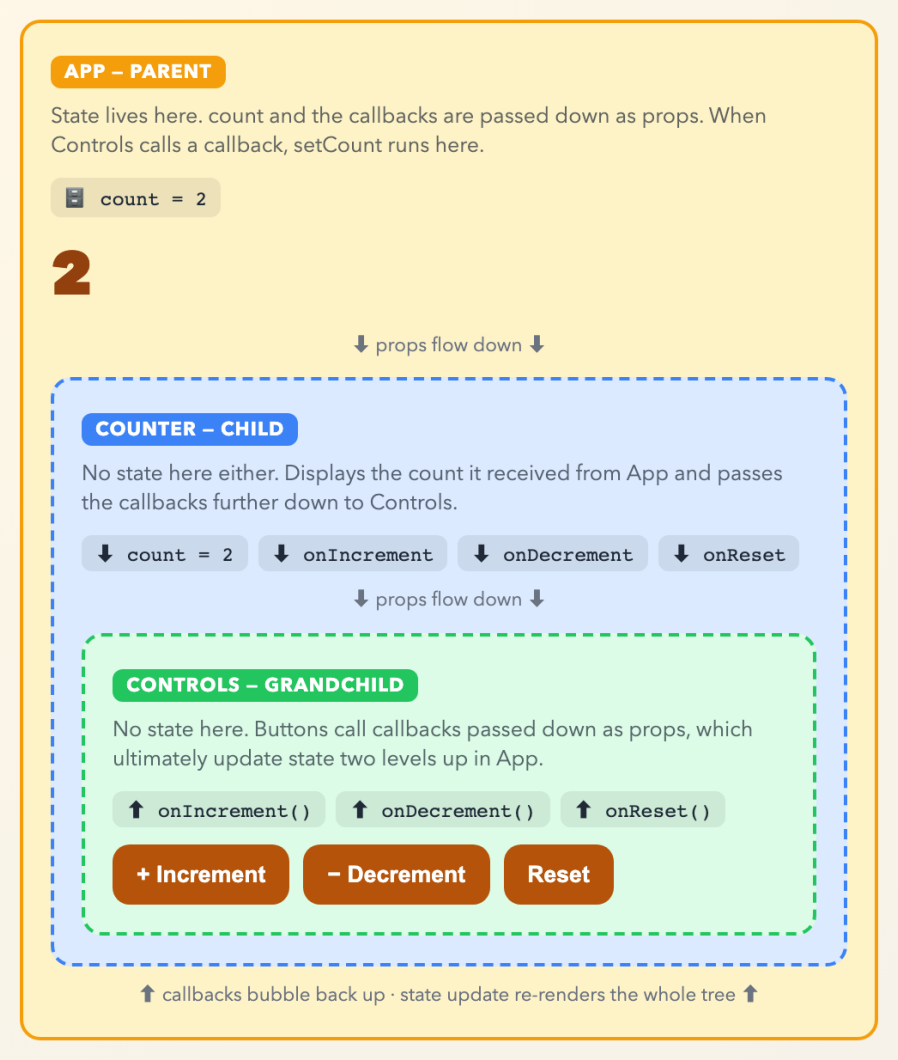


Figure 7 — lifting-state.html: App holds the state, Controls holds the buttons

2. Open `lifting-state.js` and find the only `useState` call. It is in `App`, at the top:

Component	Has state?	Gets as props	Does
`App` (parent)	yes: `count`	nothing	owns `count`, creates the three callbacks
`Counter` (child)	no	`count` and three callbacks	shows `count`, passes the callbacks on
`Controls` (grandchild)	no	three callbacks	its buttons call the callbacks

3. The buttons are in `Controls`, but the number is shown in `App` and `Counter`. So the state must live in the **closest common parent** of everything that needs it, which is `App`. This is called **lifting state up**.
4. Follow one click through the code (Figure 8). `App` creates `onIncrement={() => setCount((c) => c + 1)}` and passes it to `Counter`. `Counter` does not use it; it only passes it on to `Controls`. `Controls` puts it on a button: `<button onClick={onIncrement}>`. When you click, the function runs *in App*, so `App`'s state changes and the whole tree re-renders with the new `count`.

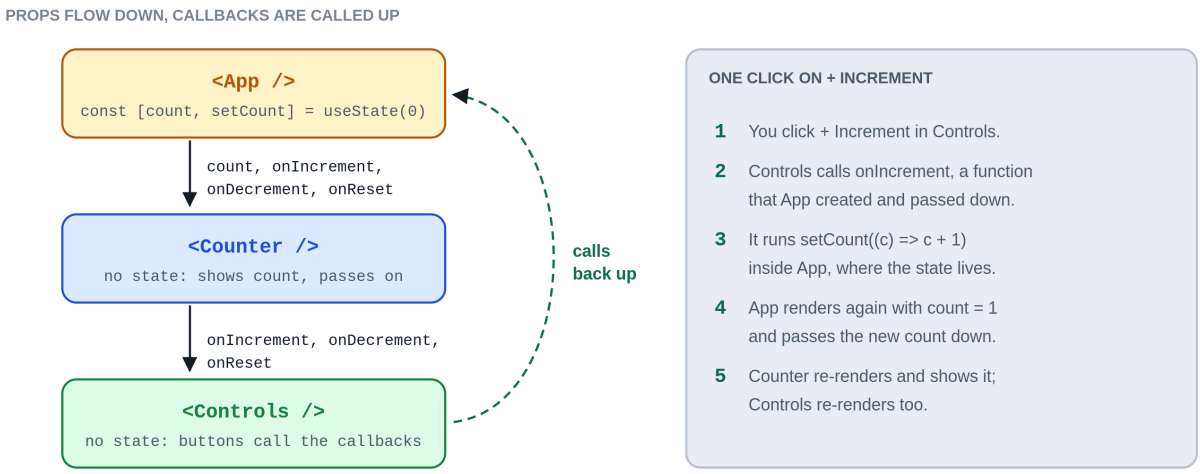


Figure 8 — data goes down as props; the click comes back up as a callback

5. Open the **Components** tab in DevTools (install the *React Developer Tools* extension if it is missing). Click `App`: under **hooks** you see **State: 0**. Click `Counter` and `Controls`: they have only **props**. Click a button and watch App's state change.

> **THINK (1 MIN)**
> What would break if you moved `useState` into `Controls`? The buttons would still work, but `App` and `Counter` could no longer show the count. A parent cannot read a child's state; data only flows down.

Step 07 — Send a new callback down the tree

Goal: a **× 2 Double** button in `Controls` that doubles the count kept in `App`.

1. In `App`, pass one more callback to `Counter`. Doubling depends on the old value, so use an updater:

```
<Counter
  count={count}
  onIncrement={() => setCount((c) => c + 1)}
  onDecrement={() => setCount((c) => c - 1)}
  onReset={() => setCount(0)}
  onDouble={() => setCount((c) => c * 2)}
/>
```

2. `Counter` must receive it and pass it on. Add `onDouble` to its parameter list and to the `<Controls>` it renders:

```
function Counter({ count, onIncrement, onDecrement, onReset, onDouble }) {
```

```
<Controls
  onIncrement={onIncrement}
  onDecrement={onDecrement}
  onReset={onReset}
  onDouble={onDouble}
/>
```

3. `Controls` receives it and gives it a button:

```
function Controls({ onIncrement, onDecrement, onReset, onDouble }) {
```

```
<button onClick={onReset}>Reset</button>
<button onClick={onDouble}>× 2 Double</button>
```

4. For the badges to match, add `<span className="prop prop-down">onDouble</span>` to Counter's badges and `<span className="prop prop-up">onDouble()</span>` to Controls' badges.
5. Save. Click Increment three times, then **× 2 Double**. The count goes from 3 to 6 (Figure 9).

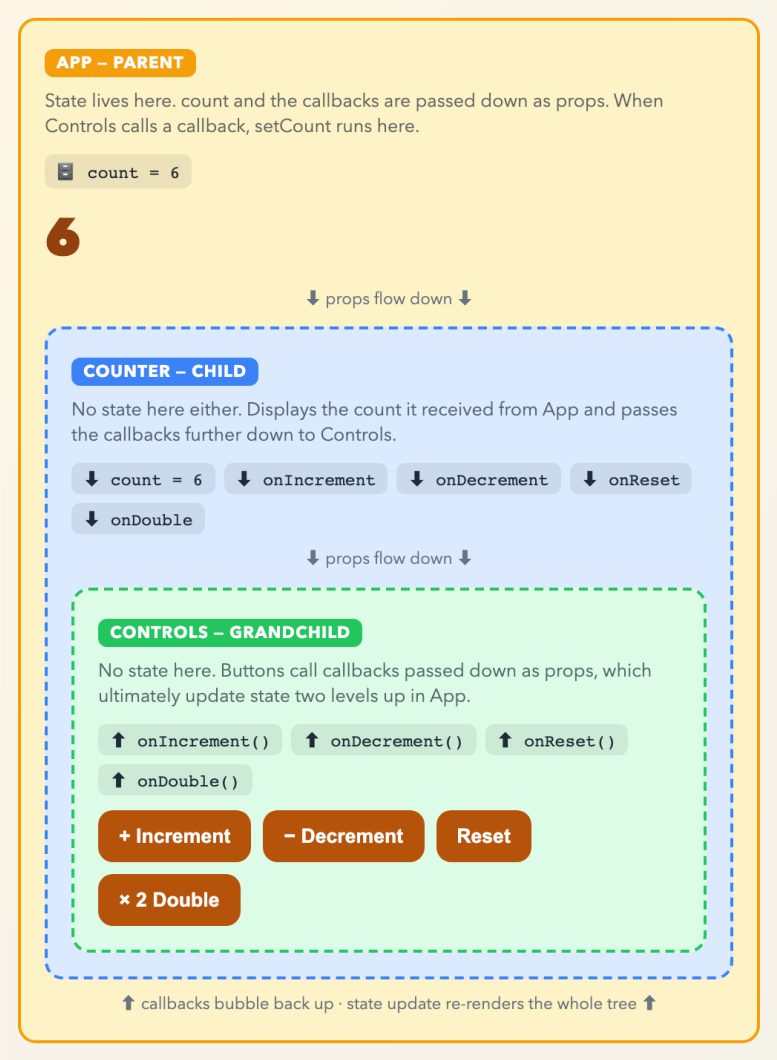


Figure 9 — the new Double button, after three Increments and one Double

> **NOTICE THE MIDDLE MAN**
> You had to edit `Counter` even though it never uses `onDouble`. Passing props through components that only forward them is called **prop drilling**. With three levels it is fine. With ten it gets tiring, and that is the problem the `useContext` hook solves later in this module.

> **CHECKPOINT 07**
> `code/checkpoints/step-07-double-button/`

Step 08 — Wrap-up, challenge and self-check

The rules of hooks

`useState` is a hook, and every hook follows two rules:

1. **Call hooks only at the top level of a component**, never inside an `if`, a loop or a nested function. React tells your `useState` calls apart by their order, so the order must be the same on every render.
2. **Call hooks only from React components** (or from your own custom hooks), not from ordinary JavaScript functions.

Arrays and objects in state

State can hold any value: a number, a string, a boolean, an array, an object. With arrays and objects, never change the old one. Make a **new** one and pass it to the setter:

Instead of…	write…
`list.push(item)`	`setList((l) => [...l, item])`
`user.age = 26`	`setUser((u) => ({ ...u, age: 26 }))`
`list.splice(i, 1)`	`setList((l) => l.filter((_, j) => j !== i))`

React compares the old and new value to decide whether to re-render. If you change the old array in place, it is still the *same* array, and React may not notice.

Challenge (optional, about 10 min)

Go back to `index.html` and `app.js`. Add a third piece of state, `log`, that starts as an empty array. Each time Increment, Decrement or Reset is clicked, add an entry such as `+5`, `−5` or `reset` to the log, and show the log as a list under the buttons with the number of clicks above it. Add a **Clear log** button. Hint: write a small helper inside `App`:

```
function record(entry) {
  setLog((l) => [...l, entry]);
}
```

A worked solution is in `code/checkpoints/challenge-click-log/`.

Self-check

☐ I can name the two things `useState` returns, and say why I never assign to the value directly.

- ☐ I can explain why `let count = 0` does not work for a counter.
- ☐ I know when to pass an updater, `setX((x) => …)`, instead of a value.
- ☐ I can explain why `setCount(count + 1)` three times in one click adds only 1.
- ☐ I can use two `useState` calls in one component and a controlled input.
- ☐ I can decide which component should own a piece of state, and pass a callback down to change it.

Summary

Idea	In code
Ask React for state	`const [count, setCount] = useState(0);`
Show it	`{count}` in the JSX
Set a fixed value	`setCount(0)`
Set from the old value	`setCount((c) => c + 1)`
More state	call `useState` again: `const [step, setStep] = useState(1);`
Share state	keep it in the closest common parent; pass the value and callbacks down as props